```
#!/usr/bin/env python3
"""
PITOOLS — DevOps фреймворк
© 2025 Герасимов Федор Анатольевич
Размер: 9434 байт | Python 3 | Linux | Персональные ЭВМ
Поддержка динамической контрольной суммы pitools.py
"""

import os
import sys
import argparse
import asyncio
import pathlib
import shlex
import paramiko
import hashlib
from pathlib import Path

# ==================== САМОПРОВЕРКА pitools (№1) ====================
# Пример MY_CHECKSUM = "1e5e080847a3f092b7c368f9ec2c5335d108a52e2edafd7c3e8d64bfe6548133"

def verify_integrity():
    """Возможность проверки SHA256 самого pitools.py при запуске"""
    hash_sha256 = hashlib.sha256()
    try:
        with open(__file__, "rb") as f:
            for chunk in iter(lambda: f.read(4096), b""):
                hash_sha256.update(chunk)
        computed = hash_sha256.hexdigest()

        # ДИНАМИЧЕСКАЯ ПРОВЕРКА: сравни с отдельным checksum.txt
        checksum_file = Path(__file__).parent / "checksum.txt"
```

```
        if checksum_file.exists():
            expected = checksum_file.read_text().strip()
            if computed != expected:
                print(f"\n⚠️  КОНТРОЛЬНАЯ СУММА pitools ≠ checksum.txt")
                print(f"   checksum.txt: {expected[:16]}...")
                print(f"   pitools.py:   {computed[:16]}...")
                print("   ⚠️  Изменения обнаружены (но работа продолжается)")
            else:
                print("🔒 pitools проверен OK ✓")
        else:
            print("ℹ️  checksum.txt не найден — проверка отключена")
    except Exception as e:
        print(f"ℹ️  Проверка пропущена: {e}")

#verify_integrity()  # ← АКТИВИРУЙТЕ при необходимости

class PITOOLS:
    """PITOOLS v6.1 — интерпретатор .prj файлов для DevOps"""

    def __init__(self, prj_file: Path):
        self.prj_file = prj_file
        self.prj_dir = prj_file.parent
        print(f"🌍 PITOOLS загружен: {self.prj_file}")

    def parse_prj(self):
        """ЛОГИКА №2: Парсит .prj → блоки деплоя"""
        blocks = []
        current = {'hosts': [], 'prj_path': None, 'files': []}

        with open(self.prj_file) as f:
            for line_num, line in enumerate(f, 1):
                line = line.strip()
```

```
            if not line or line.startswith('#'):
                continue

            parts = shlex.split(line)
            if not parts: continue
            cmd = parts[0]

            if cmd == 'MLT:':
                if current['hosts']: blocks.append(current)
                current = {
                    'hosts': [h.strip() for h in ' '.join(parts[1:]).split(',')],
                    'prj_path': None,
                    'files': []
                }
            elif cmd == 'PRJ:':
                current['prj_path'] = ' '.join(parts[1:])
            elif cmd == 'FL:':
                current['files'].append(' '.join(parts[1:]))

    if current['hosts']: blocks.append(current)
    return blocks

async def fl_deploy(self, host: str, user: str, target: str, filename: str, key: str):
    """ЛОГИКА №3: Деплой одного файла (параллельно)"""
    src = self.prj_dir / filename
    if not src.exists():
        print(f"❌ {host}: {filename} не найден")
        return False

    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
```

```
try:
    await asyncio.to_thread(
        client.connect,
        host,
        username=user,
        key_filename=key,
        timeout=10
    )
    sftp = client.open_sftp()

    # РЕКУРСИВНОЕ СОЗДАНИЕ ПАПОК
    target_dir = Path(target).parent
    parts = str(target_dir).lstrip('/').split('/')
    current_path = '/'

    for part in parts:
        if part:
            current_path += part + '/'
            try:
                sftp.stat(current_path)
            except FileNotFoundError:
                sftp.mkdir(current_path)

    # ЗАПИСЬ ФАЙЛА + ПРАВА
    remote_name = f"{host.split('.')[0]}_{Path(target).name}"
    remote_path = str(target_dir / remote_name)

    sftp.put(str(src), remote_path)
    sftp.chmod(remote_path, 0o644)
    print(f"☑ {host}: {filename} → {remote_path}")

    sftp.close()
```

```
            client.close()
            return True

        except Exception as e:
            print(f"❌ {host}: {e}")
            return False

    async def verify_deployed_files(self, host: str, user: str, files: list, key: str):
        """ЛОГИКА №5: CHECKSUM созданных файлов"""
        print(f"🔍 Checksum верификация {host}...")

        for filename in files:
            src = self.prj_dir / filename
            if src.exists():
                local_hash = hashlib.sha256(src.read_bytes()).hexdigest()

                remote_name = f"{host.split('.')[0]}_{filename}"
                cmd = f"sha256sum '{remote_name}' 2>/dev/null || echo ERROR"

                try:
                    client = paramiko.SSHClient()
                    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())
                    await asyncio.to_thread(client.connect, host, username=user, key_filename=key, timeout=5)

                    stdin, stdout, stderr = await asyncio.to_thread(
                        client.exec_command, cmd
                    )

                    remote_hash = stdout.read().decode().strip().split()[0]

                    status = "☑" if local_hash == remote_hash else "❌"
                    print(f"  {status} {filename}: {remote_hash[:8]}...")
```

```
                client.close()
            except:
                print(f"  ⚠️  {filename}: проверка недоступна")

async def deploy(self, key: str, user: str):
    """ЛОГИКА №6: Основной цикл деплоя + верификация"""
    blocks = self.parse_prj()
    print(f"🚀 PITOOLS v6.1: {self.prj_file} → {self.prj_dir}")
    print(f"📊 Блоков: {len(blocks)}")

    for i, block in enumerate(blocks, 1):
        print(f"\n🔧 БЛОК {i}: {len(block['hosts'])} хостов")

        # УБРАЛ SAFE_PATHS — деплоит везде!
        if block['prj_path']:
            print(f"📁 Цель: {block['prj_path']}")

            for fl_file in block['files']:
                print(f"📄 Деплоим: {fl_file}")

                tasks = [
                    self.fl_deploy(h, user, block['prj_path'], fl_file, key)
                    for h in block['hosts']
                ]
                results = await asyncio.gather(*tasks, return_exceptions=True)

                success = sum(1 for r in results if r is True)
                print(f"  ☑️ {success}/{len(tasks)} хостов")

                if success > 0 and block['hosts']:
                    await self.verify_deployed_files(
```

```
                    block['hosts'][0], user, [fl_file], key
                )

async def main():
    parser = argparse.ArgumentParser(
        description="""🔥 PITOOLS — DevOps .prj интерпретатор
© 2025 Герасимов Федор Анатольевич
Python 3 | Linux | Динамическая checksum""",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog="""Пример pitools.prj:
MLT: server1.ru,server2.ru
PRJ: /var/www/myapp
FL: index.html
FL: config.py""")

    parser.add_argument('prj', help='.prj файл для деплоя')
    parser.add_argument('--user', '-u', default='deploy', help='SSH пользователь')
    parser.add_argument('--key', '-k', required=True, help='Путь к SSH ключу')

    args = parser.parse_args()

    fops = PITOOLS(Path(args.prj))
    await fops.deploy(args.key, args.user)

if __name__ == '__main__':
    asyncio.run(main())
```